

UNIVERSITATEA NAȚIONALĂ DE ȘTIINȚĂ ȘI TEHNOLOGIE
POLITEHNICA DIN BUCUREȘTI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL DE CALCULATOARE



PROIECT DE DIPLOMĂ

CultureQuest - Platforma mobilă de explorare urbană bazată pe proximitate, cu recomandări personalizate prin învățare federată

Andrei Cozma

Coordonator științific:

Prof. dr. ing. Ciprian-Mihai Dobre
Prof. dr. ing. Radu-Ioan Ciobanu

BUCUREȘTI

2026

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY
POLITEHNICA OF BUCHAREST
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS
COMPUTER SCIENCE AND ENGINEERING DEPARTMENT



DIPLOMA PROJECT

CultureQuest - Proximity-Based Mobile Platform for Urban
Exploration with Personalized Recommendations via Federated
Learning

Andrei Cozma

Thesis advisor:

Prof. dr. ing. Ciprian-Mihai Dobre
Prof. dr. ing. Radu-Ioan Ciobanu

BUCHAREST

2026

CUPRINS

1	Introducere	1
1.1	Context	1
1.2	Definirea problemei	1
1.3	Obiective	2
1.4	Soluția propusă	2
1.5	Rezultatele obținute	3
1.6	Structura lucrării	3
2	Analiza Cerințelor	4
2.1	Utilizatori țintă și cazuri de utilizare	4
2.2	Cerințe funcționale	4
2.3	Cerințe non-funcționale	5
2.4	Domeniul de aplicare al proiectului	6
3	Soluții Existente	7
3.1	Aplicații existente pentru explorare urbană	7
3.2	Analiză comparativă între abordările centralizate și învățarea federată pentru recomandări	8
3.3	Învățare federată: Stadiul actual	9
3.4	Alternative respinse	10
3.5	Tehnologiile utilizate și justificarea acestora	11
3.5.1	Flutter și Riverpod	11
3.5.2	FastAPI, MongoDB și Redis	11
3.5.3	Motorul de rutare OSRM	12
3.5.4	Perceptron multi-strat	12
4	Soluția Propusă	14
4.1	Prezentare generală a sistemului	14

4.2	Arhitectura aplicației mobile	15
4.3	Arhitectura serviciilor <i>backend</i>	15
4.4	Proiectarea sistemului de învățare federată	16
4.4.1	Arhitectura modelului	16
4.4.2	Antrenarea locală	18
4.4.3	Agregarea asincronă și reducerea în funcție de vechime	18
4.4.4	Limitarea gradientilor	19
4.4.5	Confidențialitate diferențială	20
4.4.6	Controlul concurenței	20
4.5	Fluxul de generare a rutelor	21
4.6	Proiectarea elementelor de gamificare	22
4.7	Harta și navigarea	23
4.8	Arhitectura de confidențialitate	23
5	Detalii de Implementare	25
5.1	Implementarea aplicației mobile	25
5.1.1	Serviciul client de învățare federată	25
5.1.2	Adăugarea de zgomot pentru asigurarea confidențialității diferențiale	25
5.1.3	Bufferul local de interacțiuni și persistența datelor	25
5.1.4	Stocarea temporară a obiectivelor pentru funcționarea offline	25
5.1.5	Algoritmul de generare a rutelor	25
5.1.6	Sistemul de misiuni și tehnologia de <i>geofencing</i>	25
5.2	Implementarea serviciilor <i>backend</i>	25
5.2.1	Serviciul de agregare	25
5.2.2	Punctele de acces API	25
5.2.3	Sistemul de recenzii și moderarea conținutului	25
5.3	Fluxul etapei de pre-antrenare	25
5.4	<i>Framework-ul</i> pentru simularea procesului de învățare federată	25

5.5	Dificultăți întâmpinate în procesul de implementare	25
6	Evaluare	26
6.1	Corectitudinea funcțională	26
6.2	Performanța modelului de învățare federată	26
6.2.1	Rezultatele etapei de pre-antrenare	26
6.2.2	Rezultatele simulării procesului de învățare federată	26
6.3	Evaluarea confidențialității și robusteții	26
6.4	Analiză comparativă și discuții	26
7	Concluzii	27
7.1	Concluzii generale	27
7.2	Direcții de cercetare și lucrări viitoare	27
	Bibliografie	28
	Anexe	30
	Anexa A Extrase de Cod	31

SINOPSIS

CultureQuest este o platformă mobilă de explorare urbană bazată pe proximitate, oferind recomandări personalizate legate de obiectivele turistice și recreative dintr-un oraș, fără ca datele utilizatorului să părăsească dispozitivul. Problema pe care o tratez este că aplicațiile de tip *city-guide* fie oferă aceleași sugestii tuturor, indiferent de interese, fie personalizează recomandările prin salvarea centralizată a istoricului de interacțiuni cu obiectivele sau monitorizarea traseului, ceea ce ridică probleme din punctul de vedere al confidențialității. Pentru a evita acest compromis, am implementat o aplicație Flutter în care datele sunt salvate doar local, interacțiunile vizibile altor utilizatori precum recenziile sunt anonime și adăugate strict de vizitatori, iar rutele turistice generate sunt individualizate prin intermediul unei rețele neurale de tip perceptron multi-strat (MLP, *Multi-Layer Perceptron*). Modelul global inițial este pre-antrenat pe seturi de date publice, iar acesta este modificat și îmbunătățit prin tehnici de învățare federată (FL, *Federated Learning*) rulate asincron pe dispozitivele clienților. Recomandările de rute combină scorul oferit de model pentru o atracție turistică cu proximitatea acesteia, iar un sistem de misiuni gamificate încurajează explorarea și colectarea de noi date. Printr-o simulare care include mai multe acțiuni - mai exact 600 - se confirmă funcționarea fluxului FL în mod corespunzător.

ABSTRACT

CultureQuest is a proximity-based mobile urban exploration platform, offering personalized recommendations for tourist and recreational attractions in a city, without the user's data leaving the device. The problem I address is that city-guide type applications either offer the same suggestions to all users, regardless of interests, or personalize recommendations by centrally storing a user's interaction history with landmarks or by tracking their route, both of which raise privacy concerns. To avoid this trade-off, I developed a Flutter application where data is saved only locally, interactions visible to other users such as reviews are anonymous and can only be added by actual visitors and the generated tourist routes are personalized through a multi-layer perceptron (MLP) neural network. The initial global model is pre-trained on public datasets, and it is modified and improved through federated learning (FL) techniques run asynchronously on the client devices. Route recommendations combine the model's score for a tourist attraction with its proximity, while a gamified quest system encourages exploration and data collection. A simulation that includes multiple client actions - 600, to be exact - confirms that the FL flow is working properly.

1 INTRODUCERE

Acest capitol prezintă contextul în care se situează proiectul și problema care îl motivează, obiectivele propuse, o privire de ansamblu asupra soluției dezvoltate și a rezultatelor obținute, precum și modul în care este organizată restul lucrării.

1.1 Context

Telefoanele mobile au devenit, în ultimii ani, principalul instrument prin care utilizatorii interacționează cu mediul din jurul lor, iar aplicațiile mobile bazate pe locație - de la Google Maps, TripAdvisor și GetYourGuide, la diverse aplicații de tip *city-guide* - au devenit principala modalitate prin care turiștii și localnicii descoperă obiective turistice și culturale dintr-un oraș și primesc recomandări din ce în ce mai adaptate propriilor interese [6].

Pentru a oferi recomandări personalizate, aceste aplicații se bazează pe colectarea și procesarea centralizată a istoricului de interacțiuni al utilizatorului - locurile vizitate, recenziile adăugate, timpul petrecut în diverse locații - lucru care ridică probleme de confidențialitate, întrucât utilizatorul pierde controlul asupra modului în care îi sunt folosite și stocate datele legate de comportament [7].

Învățarea federată (FL, *Federated Learning*) apare ca alternativă la acest model centralizat: modelul este antrenat local, pe dispozitivul utilizatorului, folosind datele acestuia, iar către server sunt trimise doar actualizările rezultate, nu datele brute, pentru a fi agregate într-un model global. FL este deja folosită la scară destul de mare în sisteme de producție [4, 19, 9], ceea ce motivează aplicarea ei și în cadrul acestei platforme, pentru a oferi recomandări de rute turistice fără a compromite confidențialitatea clienților.

1.2 Definirea problemei

Majoritatea aplicațiilor de explorare urbană dezvoltate oferă, în absența unui profil construit din acțiunile turiștilor pe platformă, aceleași rute tuturor, indiferent de interesele fiecăruia, chiar dacă acestea sunt legate de artă, gastronomie, natură sau alte evenimente culturale. Această abordare generică este mai simplă de dezvoltat și de întreținut, însă nu ține cont de faptul că două persoane care vizitează același oraș pot avea preferințe complet diferite în ceea ce privește modul în care doresc să îl exploreze.

Varianta prin care o aplicație devine personalizată presupune, de regulă, ca istoricul de interacțiuni al utilizatorului să fie încărcat și procesat pe un server central, exact tipul de practică care aduce în discuție problemele de confidențialitate menționate mai sus. Utiliza-

torului i se prezintă, astfel, un compromis: fie acceptă recomandări generice, fie acceptă ca datele sale comportamentale să fie colectate central pentru a primi recomandări adaptate intereselor sale - iar acest compromis este punctul de plecare al proiectului de față.

1.3 Obiective

Pornind de la acest compromis, proiectul de față își propune să arate că personalizarea și păstrarea confidențialității nu se exclud reciproc, prin trei obiective principale:

- (a) O aplicație mobilă prin care utilizatorul poate descoperi obiective culturale dintr-un oraș printr-o explorare bazată pe proximitate, cu un sistem de misiuni(*quest-uri*) și puncte obținute pentru interacțiunile pe platformă, care îl încurajează să viziteze locuri noi.
- (b) Personalizarea recomandărilor de rute pentru fiecare utilizator, fără ca datele acestuia - istoricul de interacțiuni, locațiile vizitate - să părăsească dispozitivul.
- (c) Implementarea unui flux complet de FL: pre-antrenarea modelului global, ajustarea fină (*fine-tuning*), antrenarea locală prin FedAvg [13] și agregarea asincronă a actualizărilor primite de la clienți, cu garanții de confidențialitate.

1.4 Soluția propusă

Soluția dezvoltată este o aplicație mobilă realizată în Flutter, prin care utilizatorul poate vedea pe hartă obiectivele turistice și evenimentele din apropiere. Pe lângă explorare, aplicația oferă un sistem de *quest-uri*, care încurajează vizitarea unor obiective noi, dar și posibilitatea de a lăsa recenzii, povești sau chiar alte *quest-uri* pentru locurile vizitate - tot ce adaugă utilizatorii devine, în timp, parte din imaginea pe care comunitatea o are despre fiecare atracție.

Pentru personalizare, fiecare utilizator are pe dispozitiv un model mic, un perceptron multi-strat (MLP, *Multi-Layer Perceptron*) cu un strat de intrare de 22 de caracteristici, două straturi ascunse (32 și 16 neuroni) și un neuron de ieșire, care reprezintă un scor de recomandare pentru fiecare obiectiv. Pe baza acestui scor, fiecărui obiectiv i se asociază o etichetă care arată cât de recomandat este pentru utilizator, în funcție de interesele acestuia, de ora curentă, de tipul locului etc., iar la generarea rutei, scorul fiecărui obiectiv este combinat cu distanța față de poziția curentă. Modelul global inițial este pre-antrenat cu ajutorul unor seturi de date din mediul online, apoi antrenat pe dispozitiv prin FL, iar *backend-ul* (FastAPI, MongoDB, Redis) coordonează învățarea asincronă.

1.5 Rezultatele obținute

La nivel general, rezultatul este o aplicație Flutter funcțională, care reunește harta cu obiective și evenimente din apropiere, generarea de rute personalizate și sistemul de *quest-uri*. Personalizarea descrisă în secțiunea anterioară este vizibilă direct în aplicație, pentru fiecare obiectiv din apropiere și în cadrul rutelor generate.

Pe partea de FL, modelul de pe dispozitiv pornește de la o variantă a modelului global trecută și prin etapa de *fine-tuning*, iar antrenarea locală se face prin FedAvg. Agregarea pe server este asincronă, cu o reducere în funcție de vechimea (*staleness discount*) actualizărilor venite de la clienți [17] - o metodă prin care influența lor asupra modelului global este diminuată. O simulare de 600 de runde confirmă funcționarea în parametri buni a fluxului FL.

1.6 Structura lucrării

Capitolul 1 a introdus, la nivel general, contextul, problema, obiectivele, soluția propusă și rezultatele obținute. Capitolele următoare intră în detaliu pe fiecare dintre aceste aspecte.

Capitolul 2 stabilește cerințele aplicației, pornind de la cazurile de utilizare ale unui turist sau localnic care explorează un oraș, și le împarte în cerințe funcționale și non-funcționale. Tot aici se delimitează și domeniul de aplicare al proiectului.

Capitolul 3 trece în revistă aplicațiile de explorare urbană existente și modul în care acestea personalizează recomandările, comparând abordările centralizate cu FL. Se poziționează CultureQuest pe baza taxonomiei FL, se explică de ce alte alternative de algoritmi FL nu se potrivesc design-ului asincron al platformei și se justifică tehnologiile alese.

Capitolul 4 detaliază arhitectura propusă a sistemului. Aici se discută structura aplicației mobile și a serviciilor *backend*, proiectarea modelului MLP și a procesului FL, fluxul de generare a rutelor, elementele de gamificare și arhitectura de confidențialitate.

După proiectare, **Capitolul 5** trece la implementarea efectivă - serviciul client FL și mecanismul de confidențialitate diferențială (DP, *Differential Privacy*), serviciile *backend* de agregare și de moderare a conținutului, etapa de pre-antrenare și *framework-ul* folosit pentru simularea procesului FL.

Capitolul 6 evaluează rezultatele obținute - corectitudinea funcțională a aplicației, performanța modelului rezultată din pre-antrenare și din fluxul FL, efectul DP și al limitării asupra robusteții - și include o discuție comparativă față de alternativele respinse.

Capitolul 7 încheie lucrarea cu o privire de ansamblu asupra obiectivelor stabilite inițial, în raport cu ce a fost dezvoltat și propune direcții viitoare - de exemplu, personalizarea suplimentară per utilizator sau alte tipuri de agregare în ceea ce privește modelul rețelei neurale.

2 ANALIZA CERINȚELOR

Capitolul de față detaliază cerințele care reies din obiectivele stabilite anterior, și anume cine sunt utilizatorii și care sunt cazurile de utilizare care trebuie acoperite, ce funcționalități rezultă de aici, ce constrângeri non-funcționale se fac necesare și ce rămâne în afara domeniului de aplicare. Toate aceste cerințe stau la baza deciziilor de proiectare din Capitolul 4, fiind verificate ulterior în evaluarea din Capitolul 6.

2.1 Utilizatori țintă și cazuri de utilizare

Utilizatorul principal al aplicației este un turist sau un localnic care explorează un oraș - sau o zonă a acestuia - cu dorința de a vizita obiective turistice sau culturale din proximitate, adaptate intereselor proprii. Primul caz de utilizare este cel de generare de rute și suport pe parcursul urmăririi acestora, în care utilizatorul vede pe hartă poziția sa curentă, locurile de interes din apropiere și primește o rută care combină preferințele sale cu acestea. Mai mult, interacțiunile cu aplicația - vizite, (*quest-uri*) finalizate, recenzii - determină ajustarea, prin FL, a modelului de pe dispozitiv - și, implicit, și a celui global - fără ca istoricul brut al interacțiunilor să ajungă pe server. Un al doilea caz de utilizare este cel de contribuție de conținut, în care utilizatorul poate propune obiective noi, povești, *quest-uri* sau recenzii, toate acestea necesitând aprobarea de către alte persoane care folosesc aplicația sau de către un administrator. Prin conturile de tip administrator, relevante mai ales pentru 2.2, se aprobă sau se respinge conținutul propus de utilizatorii obișnuiți.

2.2 Cerințe funcționale

Pornind de la cazurile de utilizare din secțiunea anterioară, cerințele funcționale au fost sintetizate în tabelul 1, fiecare cu o descriere și secțiunile din Capitolele 4-6 unde cerința este tratată. Primul grup (de la **FR-1** la **FR-4**) sumarizează bucla principală de explorare: afișarea pe hartă a obiectivelor turistice din proximitate, generarea rutei care combină scorul modelului de tip MLP cu distanța față de poziția curentă, *quest-urile* disponibile la obiectivele vizitate și posibilitatea de a adăuga sau vizualiza recenzii.

Al doilea grup (de la **FR-5** la **FR-7**) ține de personalizare și administrare: **FR-5** descrie fluxul procesului FL - primirea parametrilor modelului actualizat de la server, urmată de antrenarea locală, fără ca istoricul interacțiunilor să ajungă pe server, **FR-6** descrie controalele de confidențialitate - asigurarea DP, vizibilitatea asupra datelor, iar **FR-7** descrie rolul de administrator - aprobarea sau respingerea conținutului propus.

Tabelul 1: Cerințe funcționale

ID	Descriere	Secțiune
FR-1	Obiective din proximitate pe hartă.	4.7, 5.1.4, 6.1
FR-2	Rute personalizate: scor model MLP și proximitate.	4.5, 5.1.5, 6.1
FR-3	<i>Quest-uri</i> la obiectivele vizitate; puncte și progres.	4.6, 5.1.6, 6.1
FR-4	Adăugare și vizualizare recenzii pentru obiective.	5.2.3, 6.1
FR-5	Sincronizare model și antrenare locală, fără date colectate pe server.	4.4, de la 5.1.1 la 5.1.3, 5.2.1, 6.2
FR-6	Zgomot DP aplicat pe actualizări; vizibilitate asupra datelor trimise.	4.8, 5.1.2, 6.3
FR-7	Administrator: aprobă sau respinge conținut propus (obiective, povești, <i>quest-uri</i> , recenzii).	5.2.3, 6.1

2.3 Cerințe non-funcționale

La fel ca în secțiunea 2.2, cerințele non-funcționale au fost sintetizate în tabelul 2, cu aceeași structură (descriere și secțiunile din Capitolele 4-6 unde sunt tratate). Primul grup (de la **NFR-1** la **NFR-3**) are legătură cu constrângerile impuse pe dispozitiv: **NFR-1** cere ca nicio informație privată despre utilizator - exceptând lista de interese - să nu părăsească telefonul, **NFR-2** cere ca obiectivele turistice să rămână disponibile și fără conexiune la internet, iar **NFR-3** cere ca antrenarea locală să nu fie complexă din punct de vedere computațional pentru un telefon obișnuit - ceea ce a motivat alegerea unui model MLP de dimensiuni reduse.

Al doilea grup ține de partea de *backend*: **NFR-4** impune ca serviciul de agregare să suporte actualizări asincrone primite de la mai mulți clienți - fără conflicte între runde, iar **NFR-5** impune ca utilizatorii să primească recomandări utile încă de la început printr-un model global pre-antrenat [4] - și din caracteristici bazate pe ariile de interes asociate fiecărui obiectiv - până când procesul FL își face efectul.

Tabelul 2: Cerințe non-funcționale

ID	Descriere	Secțiune
NFR-1	Confidențialitate: nicio interacțiune a utilizatorului nu părăsește dispozitivul.	4.8, de la 5.1.2 la 5.1.3, 6.3
NFR-2	Reziliență offline: obiective turistice stocate în <i>cache</i> local.	5.1.4, 6.1
NFR-3	Cost redus de antrenare pe dispozitiv.	4.4.1, 5.1.1, 6.2.1
NFR-4	Scalabilitatea <i>backend-ului</i> la actualizările asincrone.	4.4.6, 5.2.1, 6.2.2
NFR-5	<i>Cold start</i> : recomandări utile pentru utilizatori folosind modelul pre-antrenat.	4.4.1, 5.3, 6.2.1

2.4 Domeniul de aplicare al proiectului

Cerințele funcționale și non-funcționale stabilite anterior delimitează domeniul de aplicare al proiectului: aplicația mobilă cu recomandări de rute turistice, pornind de la pre-antrenarea globală și fiind îmbunătățite prin FL. Rămâne în afara domeniului de aplicare personalizarea suplimentară la nivel de utilizator prin straturi separate ale modelului MLP (*personal head*), care nu ar fi trimise către server, rezultând într-o arhitectură de tip FedPer [3] - propusă ca direcție de lucrări viitoare, dar neimplementată în versiunea curentă.

Având stabilite cerințele și limitele proiectului, Capitolul 3 trece în revistă soluțiile existente pentru explorare urbană (Google Maps, TripAdvisor, alte aplicații de tip *city-guide*) și realizează o analiză comparativă între abordările centralizate și cele bazate pe FL pentru recomandări.

3 SOLUȚII EXISTENTE

Înainte de proiectarea propriu-zisă a soluției, acest capitol analizează contextul în care se înscrie CultureQuest. Aplicațiile de explorare urbană existente oferă un grad bun de personalizare a recomandărilor, însă acestea se bazează pe sisteme centralizate de colectare a datelor, de unde și comparația cu abordările bazate pe FL și cu stadiul actual al cercetării din domeniu, folosit pentru a poziționa CultureQuest în cadrul soluțiilor FL. Pe baza acestei analize, capitolul explică de ce o parte dintre cele mai cunoscute variante de algoritmi FL nu se potrivesc design-ului asincron al platformei și justifică restul tehnologiilor folosite în implementare.

3.1 Aplicații existente pentru explorare urbană

Platformele mobile de recomandare pentru turiști [6] oferă un anumit nivel de personalizare bazat pe istoricul de activitate al utilizatorului, stocat și procesat pe serverele furnizorului. Google Maps generează, de exemplu, recomandări pe baza istoricului de locații, a căutărilor și a evaluărilor asociate contului Google¹. TripAdvisor merge pe o logică asemănătoare, prin faptul că sugestiile de obiective și restaurante sunt adaptate pe baza recenziilor citite, a căutărilor anterioare și a locațiilor vizitate, tot prin contul de utilizator, procesat central². În ambele cazuri, personalizarea funcționează doar pentru că acest istoric de interacțiuni ajunge să fie analizat pe server, nu pe dispozitivul utilizatorului.

Pe lângă aceste platforme axate pe navigare, există și aplicații specializate în activități turistice legate de rezervări, precum GetYourGuide, care recomandă tururi cu ghizi, bilete și experiențe pe baza destinației căutate și a istoricului de căutări sau rezervări al utilizatorului, tot stocat și procesat central³. Spre deosebire de Google Maps sau TripAdvisor, accentul este pus pe activități plătite și predefinite, nu pe explorarea liberă a orașului, așa că nu există rute generate dinamic în funcție de poziția utilizatorului. Dincolo de aceste diferențe, toate trei se bazează pe același principiu - ML centralizat, antrenat pe date de la un număr mare de utilizatori.

La polul opus se situează aplicațiile axate pe confidențialitate, precum Organic Maps, un proiect *open-source* bazat pe date de la OpenStreetMap, fără cont de utilizator și fără colectare de date⁴. Aici personalizarea nu există, iar toți utilizatorii dintr-o zonă văd același set de obiective, indiferent de interese, pentru că nu există niciun istoric din care acestea să

¹Google, pagina oficială despre recomandările personalizate. <https://support.google.com/websearch/answer/17025248?hl=en> (accesat 19.06.2026).

²TriAdvisor, pagina oficială despre funcționarea recomandărilor/politica de confidențialitate. <https://tripadvisor.mediaroom.com/US-privacy-policy> (accesat 19.06.2026).

³GetYourGuide, pagina oficială despre cum funcționează recomandările și politica de confidențialitate. <https://www.getyourguide.com/c/privacy-policy> (accesat 19.06.2026).

⁴Organic Maps, pagina oficială - „no tracking, no data collection”. <https://organicmaps.app/privacy/> (accesat 19.06.2026).

poată fi accesate. Între cele două extreme - fie personalizare cu date colectate central, fie confidențialitate fără personalizare - se situează CultureQuest, unde scorul de personalizare este calculat prin FL, fără ca datele brute să părăsească dispozitivul. Toate aceste cazuri sunt adunate în tabelul 3.

Tabelul 3: Comparație între aplicații existente de explorare urbană și CultureQuest

Aplicație	Personalizare	Colectare date	ML local	Gamificare
Google Maps	Da	Centralizat	Nu	Nu
TripAdvisor	Da	Centralizat	Nu	Limitată
GetYourGuide	Da	Centralizat	Nu	Nu
Organic Maps	Nu	Fără	Nu	Nu
CultureQuest	Da	Local	Da	Da

3.2 Analiză comparativă între abordările centralizate și învățarea federată pentru recomandări

Sistemele de recomandare clasice se bazează, în general, pe filtrarea colaborativă, care recomandă unui utilizator ceea ce au apreciat alți utilizatori cu interese similare, și pe filtrarea bazată pe conținut, care recomandă obiective similare cu cele apreciate deja de același utilizator [2]. În ambele cazuri, determinarea similarităților are nevoie de o bază de date centralizată cu istoricul de activitate al clienților - exact modelul din spatele recomandărilor descrise în secțiunea anterioară pentru aplicațiile menționate. Avantajul e accesul la un volum mare de date eterogene, din care modelul învață tendințe generale utile chiar și pentru utilizatori noi, pe baza unor profiluri asemănătoare existente în sistem. Prețul acestui avantaj este însă tot cel discutat mai sus. Pentru ca modelul să poată face aceste calcule, istoricul de activitate al fiecărui utilizator trebuie să ajungă și să rămână pe server, sub controlul furnizorului.

FL schimbă direcția, astfel încât modelul ajunge la date, nu invers - antrenarea are loc local, pe dispozitivul fiecărui utilizator, iar pe server ajung doar actualizările rezultate ale modelului, nu istoricul de activitate în sine. Datele rămân astfel pe dispozitiv, iar confidențialitatea nu mai depinde de politicile furnizorului, ci de proiectarea protocolului de agregare. Diferența se vede cel mai clar la pornirea la rece (*cold-start*). O aplicație aflată la început nu are încă un model antrenat suficient din cauza unui număr mic de persoane care o folosesc, așa că recomandările vin dintr-un model global pre-antrenat și din caracteristicile bazate pe conținut ale obiectivelor, la fel ca în cazul centralizat - personalizarea prin FL se construiește abia pe măsură ce utilizatorii interacționează cu aplicația.

3.3 Învățare federată: Stadiul actual

FL a apărut ca direcție de cercetare odată cu FedAvg [13], ca răspuns direct la problema discutată în secțiunea anterioară - cum poate fi antrenat un model pe datele mai multor clienți, fără ca acestea să fie centralizate pe un server. De atunci, domeniul s-a diversificat, acoperind variante de agregare, metode DP, robustețe la clienți adversariali și scenarii de personalizare [7]. Pentru a situa CultureQuest în acest cadru, ne putem ghida după două axe de clasificare devenite standard în literatură: tipul de FL, în funcție de cum sunt distribuite datele între clienți, și modul de desfășurare, în funcție de cine sunt clienții și care este comportamentul lor.

Prima axă, propusă de Yang et al. [18], împarte FL în trei categorii:

- **Orizontal (HFL)** - clienții au același spațiu de caracteristici, dar mulțimi diferite de exemple, fiind situația tipică atunci când același tip de date este colectat de la utilizatori diferiți.
- **Vertical (VFL)** - clienții au în comun o parte din exemple, dar caracteristici diferite, ca în cazul unor organizații care dețin date diferite despre aceiași utilizatori.
- **FL cu transfer (FTL)** - pentru situațiile în care nici exemplele, nici caracteristicile nu se suprapun semnificativ.

CultureQuest se încadrează clar în categoria HFL. Fiecare telefon antrenează același model, pe același spațiu de 22 de caracteristici asociate obiectivelor turistice, diferența dintre clienți fiind doar istoricul propriu de interacțiuni.

A doua axă împarte FL în două moduri de desfășurare [7, 10]:

- **Cross-device** - clienții sunt un număr mare de dispozitive individuale, cu conexiuni nesigure și resurse limitate, fiecare contribuind cu o cantitate mică de date.
- **Cross-silo** - clienții sunt un număr mic de organizații sau servere, cu conexiuni stabile și seturi de date mari.

CultureQuest este, din această perspectivă, un sistem *cross-device*. Telefoanele utilizatorilor sunt conectate la internet intermitent, fiecare cu un volum mic de interacțiuni locale. Un sistem *cross-device* are nevoie de agregare asincronă [17], pentru că acești clienți nu pot fi sincronizați la fiecare rundă de învățare, și de mecanisme de robustețe la actualizări extreme [12]. Figura 1 situează CultureQuest pe ambele axe.

	HFL	VFL	FTL
Cross-Device	CultureQuest		
Cross-Silo			

Figura 1: Poziționarea CultureQuest în taxonomia FL, pe axa tipului de FL (HFL/VFL/FTL) și axa modului de desfășurare (Cross-Device/Cross-Silo). Adaptat din Yang et al. 2019 [18] și Kairouz et al. 2021 [7].

3.4 Alternative respinse

Eterogenitatea datelor între clienți este una dintre provocările cele mai notabile ale FL [10], iar o direcție importantă de cercetare propune algoritmi care corectează deriva (*drift*) dintre actualizările locale ale clienților și modelul global. FedProx [11] adaugă funcției de cost locale un termen proximal, care penalizează distanța dintre ponderile actualizate și cele ale modelului global de la începutul runde de învățare. SCAFFOLD [8] introduce, pentru fiecare client, o variabilă de control care estimează cât de mult diferă actualizarea sa de media cohorței și corectează gradientul local pe baza acestei estimări. FedDyn [1] ajunge la un rezultat similar printr-un regularizator dinamic, recalculat la fiecare rundă pe baza istoricului actualizărilor clientului. Toate cele trei leagă actualizarea unui client de comportamentul celorlalți clienți selectați în aceeași rundă.

Această legătură nu are însă un corespondent în design-ul CultureQuest, unde fiecare rundă agregă modelul global cu actualizarea unui singur client. Nu există o cohortă de la care un client să se abată, nici actualizări paralele de reconciliat la final de rundă - condiții pe care se bazează atât termenul proximal din FedProx, cât și variabilele de control din SCAFFOLD sau regularizatorul din FedDyn. Aplicat în acest context, un astfel de termen de corecție ar trage actualizarea locală către un punct de referință arbitrar, fără ca vreo garanție de convergență din literatură să mai fie valabilă, întrucât toate cele trei presupun, în demonstrațiile lor, runde sincrone cu mai mulți clienți. De aceea, design-ul asincron al CultureQuest are nevoie de altceva - un mecanism care reduce influența actualizărilor venite de la clienți cu un model local prea vechi (FedAsync, menționat în secțiunea anterioară). Tabelul 4 rezumă această comparație.

Tabelul 4: Comparație între FedAvg și algoritmi cu corecție explicită a derivatei (FedProx, SCAFFOLD, FedDyn), și soluția adoptată de CultureQuest.

Algoritm	Sincron?	Mecanism de corecție a derivatei	1 client/rundă?
FedAvg	Da	-	Da
FedProx [11]	Da	Termen proximal (penalizare față de modelul global)	Nu
SCAFFOLD [8]	Da	Variabilă de control per client	Nu
FedDyn [1]	Da	Regularizator dinamic per client	Nu
CultureQuest	Nu	Reducere în funcție de vechime	Da

3.5 Tehnologiile utilizate și justificarea acestora

Alegerea tehnologiilor pentru CultureQuest a fost ghidată de câteva criterii comune, precum compatibilitatea *cross-platform* pentru aplicația mobilă, ecosistemul care se integrează natural cu componenta FL, cu inferența modelului și cu soluții *open-source* care pot fi găzduite local. În secțiunile următoare se descriu aceste alegeri, componentă cu componentă.

3.5.1 Flutter și Riverpod

Pentru aplicația mobilă, CultureQuest folosește Flutter⁵ și Riverpod⁶ pentru gestionarea stării. Flutter are avantajul de a permite o singură bază de cod Dart pentru Android și iOS, dar și partea de *hot reload* care accelerează dezvoltarea de cod. Alternativele posibile sunt fie React Native, cu un *bridge* între codul de JavaScript și componentele native, adică cu adăugarea unui mecanism bidirecțional de comunicare între cele două - deci, un strat de complexitate suplimentar - fie dezvoltare nativă pe cele două platforme, caz în care logica clientului FL s-ar duplica. În ceea ce privește Riverpod, acesta oferă un model de gestionare a stării declarativ și testabil. Este, prin urmare, potrivit pentru o aplicație în care starea ecranului curent trebuie sincronizată cu procese care rulează în fundal, precum antrenarea locală a modelului.

3.5.2 FastAPI, MongoDB și Redis

Pe partea de *backend*, CultureQuest folosește FastAPI⁷, MongoDB⁸ și Redis⁹. FastAPI este scris în Python, iar asta înseamnă că face posibilă integrarea directă a serviciilor de agregare

⁵Documentația oficială Flutter. <https://flutter.dev> (accesat 19.06.2026).

⁶Documentația oficială Riverpod. <https://riverpod.dev> (accesat 19.06.2026).

⁷Documentația oficială FastAPI. <https://fastapi.tiangolo.com> (accesat 19.06.2026).

⁸Documentația oficială MongoDB. <https://www.mongodb.com/docs/> (accesat 19.06.2026).

⁹Documentația oficială Redis. <https://redis.io/docs/> (accesat 19.06.2026).

FL și a antrenării modelului MLP cu ecosistemul PyTorch folosit și la pre-antrenare, din nou, fără introducerea unui strat de complexitate între limbaje. Suportul nativ pentru operații asincrone se potrivește cu un număr mare de clienți conectați intermitent, ceea ce este tipic pentru un sistem *cross-device*. Legat de MongoDB, acesta stochează date variabile - obiective, *quest-uri*, povești, recenzii - într-un mod mai flexibil decât ar permite folosirea unei baze de date relaționale. Redis, la rândul lui, acoperă starea nepersistentă a procesului FL, deoarece stocarea în memorie este suficientă, dar și mai rapidă.

3.5.3 Motorul de rutare OSRM

Funcționalitatea de rutare se bazează pe OSRM (*Open Source Routing Machine*)¹⁰, un motor *open-source*. Poate fi găzduit local, pe același server cu restul serviciilor de *backend*, fără dependență de un API extern plătit. OSRM calculează rute pe rețeaua de drumuri pe baza datelor OpenStreetMap și răspunde suficient de rapid pentru a fi folosit de utilizator pe parcursul unei rute.

3.5.4 Perceptron multi-strat

Modelul de recomandare al CultureQuest este un MLP. Arhitectura este simplă: un număr mic de straturi complet conectate, pornind de la cele 22 de caracteristici asociate obiectivelor turistice și culturale. Pe dispozitiv, modelul nu este folosit doar pentru antrenarea locală, ci și pentru afișare - de fiecare dată când utilizatorul deschide pagina cu informații a unui obiectiv turistic, aplicația face o nouă trecere *forward pass* prin model, ca să calculeze scorul de potrivire afișat pentru turist. Tocmai pentru că rulează la fiecare interacțiune, modelul trebuie să fie o implementare proprie, cât mai ușoară și fără dependențe externe.

Alternativele cunoscute pentru rularea unui model de rețea neuronală pe un dispozitiv sunt TFLite¹¹ și PyTorch Mobile¹², ambele *runtime-uri* dedicate, cu dependențe native pentru fiecare platformă. Pentru un MLP de dimensiuni reduse precum ale celui folosit în cazul de față, aceste *runtime-uri* ar crește mărimea aplicației cu zeci de *megabytes* doar pentru a executa câteva înmulțiri de matrice, fără avantaje vizibile. Implementarea în Dart rămâne, deci, mai mică, fără dependențe de *runtime* suplimentare, și compatibilă nativ cu Flutter, după cum reiese și din tabelul 5.

¹⁰Documentația oficială OSRM. <https://project-osrm.org> (accesat 19.06.2026).

¹¹Documentația oficială TensorFlow Lite. <https://www.tensorflow.org/lite> (accesat 19.06.2026).

¹²PyTorch Mobile este parțial înlocuit de ExecuTorch; documentația oficială ExecuTorch. <https://docs.pytorch.org/executorch/stable/index.html> (accesat 19.06.2026).

Tabelul 5: Comparație între implementarea MLP proprie și runtime-urile dedicate (TFLite, PyTorch Mobile) pentru inferență pe dispozitiv.

Implementare	Dimensiune model	Dependențe de runtime	Compatibil cu Flutter
MLP propriu (Dart)	Redusă - doar parametrii modelului	Niciuna	Da, cod nativ Dart
TFLite	Model + runtime (zeci de MB)	Bibliotecă nativă per platformă	Da, prin plugin
PyTorch Mobile	Model + runtime (zeci-sute de MB)	Bibliotecă nativă per platformă	Da, prin plugin

4 SOLUȚIA PROPUȘĂ

Acest capitol detaliază proiectarea CultureQuest, cu suportul a câtorva diagrame, tabele și ecuații care justifică deciziile de design. Pornește de la o privire de ansamblu asupra arhitecturii, după care urmează aplicația mobilă și serviciile *backend*, fiecare cu propria diagramă de structură. Cea mai mare parte a capitolului este dedicată proiectării sistemului FL (4.4). Urmează fluxul de generare a rutelor, elementele de gamificare și harta cu navigarea, iar în final capitolul se încheie cu arhitectura de confidențialitate, care descrie deciziile de proiectare legate de separarea datelor între dispozitiv și server.

4.1 Prezentare generală a sistemului

La nivel general, CultureQuest este format din patru componente: aplicația mobilă, scrisă în Flutter, un *backend* FastAPI, o bază de date MongoDB pentru date persistente și o instanță Redis pentru starea nepersistentă a procesului FL. Figura 2 arată comunicarea dintre ele. Aplicația mobilă este singurul punct de legătură dintre utilizator și sistem - toate cererile trec prin *backend*, care, în funcție de tipul de date, citește sau scrie în MongoDB sau în Redis.

Diagrama evidențiază trei fluxuri principale, pe care le-am numerotat cu (1), (2) și (3). Primul (1), cererea de hartă și rute, funcționează astfel: aplicația mobilă trimite poziția curentă, *backend-ul* interoghează baza de date pentru obiectivele din apropiere, le oferă un scor prin intermediul modelului FL global ținut în Redis și calculează, pe baza scorurilor, o rută pe care o trimite înapoi. Al doilea (2), agregarea FL, funcționează similar: dispozitivul trimite actualizarea modelului antrenat local, *backend-ul* o agregă cu ajutorul stării ținute în Redis și trimite înapoi modelul global actualizat. Recenziile, poveștile și *quest-urile* adăugate de utilizatori formează al treilea flux (3), unde acestea trec printr-un pas de moderare înainte să fie salvate în MongoDB.

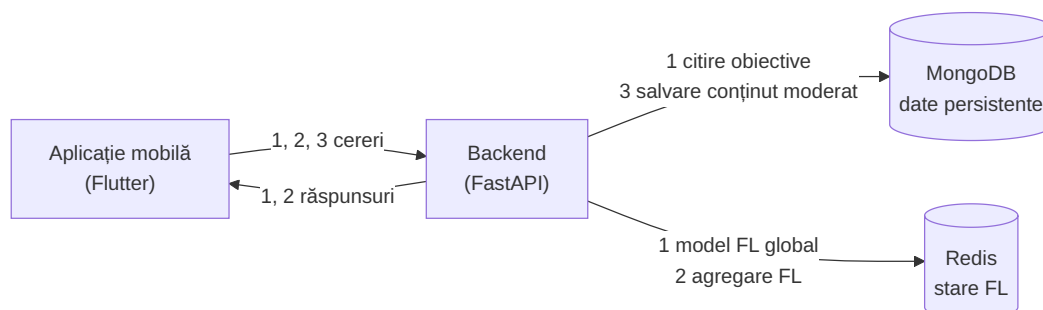


Figura 2: Arhitectura generală a sistemului CultureQuest: aplicația mobilă comunică exclusiv cu *backend-ul*, care citește sau scrie în MongoDB și Redis.

4.2 Arhitectura aplicației mobile

Aplicația mobilă este organizată pe module de tip *feature*, unde fiecare funcționalitate principală este asociată unui director propriu cu ecrane și provideri, fără dependențe directe față de alte module. Cele șapte module sunt *map* - pentru hartă și obiectivele de pe aceasta, dar și pentru generare de rute, *auth* - pentru autentificare, *federated* - pentru clientul FL, *onboarding* - pentru alegerea intereselor de către utilizator la instalarea aplicației, *profile* - pentru profilul utilizatorului și statusul procesului FL, *proximity* - pentru partea *geofencing* și detectarea proximității față de obiective) și *quest* - pentru sistemul de misiuni.

Gestionarea stării intră în responsabilitatea Riverpod, care leagă modulele între ele prin provideri declarativi. Aceștia sunt disponibili de la orice nivel al arborelui de *widget-uri*, rezultând avantajul că datele nu mai sunt transmise manual prin constructori. Figura 3 ilustrează arhitectura în trei straturi. Modulele de tip *feature* folosesc provideri Riverpod specializați, care la rândul lor delegă atât cererile către *backend*, cât și operațiile de stocare locală altor servicii dedicate. Providerii partajați arată dependențele funcționale dintre module: *fl_provider* este utilizat de *map* (pentru scorul de potrivire afișat în cadrul unui obiectiv), de *federated* (pentru statusul agregării FL) și de *profile* (pentru statusul antrenării locale). De asemenea, *proximity_provider* este partajat de *proximity* și *quest*, cel din urmă depinzând de evenimentele de *geofencing* pentru a permite completarea *quest-urilor* disponibile pentru un obiectiv.

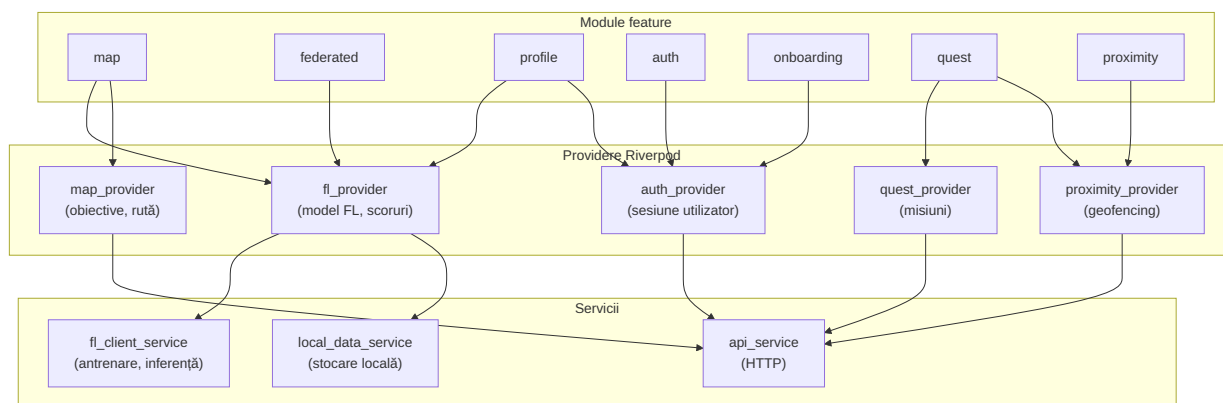


Figura 3: Arhitectura aplicației mobile CultureQuest: modulele de tip *feature* folosesc provideri Riverpod partajați.

4.3 Arhitectura serviciilor *backend*

Backend-ul aplicației este implementat cu FastAPI și structurat pe domenii de funcționalitate. Fiecare colecție MongoDB are un grup de rute dedicat și un serviciu de logică de business asociat. Grupurile de rute sunt împărțite în felul următor: un grup pentru autentificarea utilizatorilor (*auth*, cu înregistrare și autentificare pe bază de token JWT), obiectivele turistice și

culturale (*landmarks*), misiunile (*quests*), rutele turistice (*routes*), evenimentele (*events*), recenziiile (*comments*), validarea comunitară a conținutului (*community*) și schimbul de ponderi FL cu clienții (*federated*).

Modelul de date - a cărui schemă completă este ilustrată în figura 8 din Anexe - este construit în jurul colecțiilor **USERS** și **LANDMARKS**: USERS stochează profilul utilizatorului, format din credențialele acestuia și lista de interese pe care le poate selecta, și care intră ulterior în construirea vectorului de caracteristici pentru modelul FL, iar LANDMARKS reprezintă obiectivele care au drept caracteristică principală locația în format GeoJSON indexată 2dsphere, necesară interogărilor de proximitate. De colecția LANDMARKS sunt legate direct **QUESTS**, **EVENTS**, **COMMENTS**, **STORIES** și **VISITS**, dar și **ROUTES** pentru stocharea rutelor ca secvențe ordonate de referințe către LANDMARKS (fie rute globale pre-generate, fie generate de utilizatorii care doresc să parcurgă un traseu cu obiectivele turistice din apropiere).

Conținutul generat de utilizatori trece printr-un flux de moderare înainte de publicare. *Questurile* propuse și poveștile sunt create cu statusul *pending* și devin disponibile altor utilizatori doar după aprobarea de către un administrator sau de către cinci alți utilizatori. Comentariile sunt, însă, publicate imediat, dar pot fi marcate automat prin câmpul *flagged* de modulul de moderare a conținutului, acestea ajungând să fie vizibile conturilor de tip administrator pentru a fi revizuite manual.

4.4 Proiectarea sistemului de învățare federată

Sistemul FL din CultureQuest este proiectat să asocieze un scor de potrivire fiecărui obiectiv și să personalizeze generarea rutelor fără ca datele de interacțiune să părăsească dispozitivul utilizatorului. Fluxul FL este structurat în șase componente: un model MLP de dimensiuni reduse care rulează pe dispozitiv, o procedură de antrenare locală bazată pe FedAvg, un mecanism de agregare asincronă cu reducere pe bază de vechime conform FedAsync, limitarea normei actualizărilor agregate pentru ca modelul să nu fie afectat de contribuțiile extreme, adăugarea de zgomot Gaussian pentru garantarea DP, dar și un mecanism de control al concurenței bazat pe Redis.

4.4.1 Arhitectura modelului

Modelul folosit pentru personalizarea din cadrul aplicației este un MLP cu arhitectura $22 \rightarrow 32 \rightarrow 16 \rightarrow 1$, model ilustrat în figura 4, în care se descrie faptul că straturile ascunse folosesc funcția de activare ReLU, pe când stratul de ieșire aplică funcția sigmoid, producând un scor continuu în $[0, 1]$ care determină gradul de potrivire dintre profilul utilizatorului și un obiectiv dat. Modelul însumează 1.281 de parametri, acest număr mic de parametri fiind ales deliberat pentru a menține costul computațional redus pe dispozitiv. Acesta rulează atât în antrenare,

cât și la fiecare deschidere a fișei unui obiectiv.

Vectorul de intrare de 22 de dimensiuni grupează caracteristicile în patru categorii, iar fiecare caracteristică este detaliată în tabelul 6. Primele 14 dimensiuni codifică profilul de interese al utilizatorului și tipul obiectivului în format *one-hot*. Contextul temporal (dimensiunile 14-16) stabilește disponibilitatea obiectivului la momentul interogării, dacă la momentul actual este weekend și mai include ora normalizată la $[0, 1]$. Contextul spațial și de rută (dimensiunile 17-21) includ rangul relativ de distanță față de alte posibile obiective care pot fi alese pentru rută, scorul de potrivire a intereselor și trei caracteristici care sunt active doar în etapa de generare a rutelor.

Același model rulează pentru fiecare locație în parte, cu un vector de caracteristici construit specific acesteia, scorul rezultat fiind convertit într-o categorie de potrivire afișată pe fișa obiectivului - **scăzut** pentru scoruri sub 0,4, **mediu** pentru intervalul $[0,4, 0,7)$ și **ridicat** pentru scoruri mai mari sau egale decât 0,7.

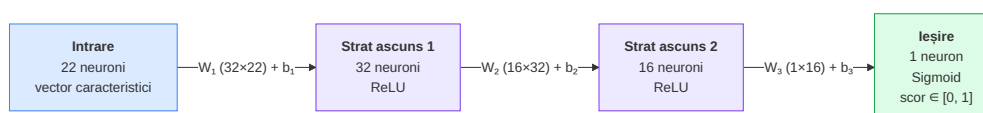


Figura 4: Arhitectura modelului MLP utilizat pentru personalizare.

Tabelul 6: Vectorul de caracteristici de intrare al modelului FL (22 dimensiuni).

Dim.	Caracteristică	Tip	Descriere
0–5	Interesele utilizatorului	one-hot	artă, arhitectură, istorie, gastronomie, natură, muzică
6–13	Tipul obiectivului	one-hot	muzeu, monument, parc, galerie, restaurant, piață, clădire, altele
14	isOpen	binar	obiectivul este deschis
15	isWeekend	binar	ziua curentă este sâmbătă sau duminică
16	oră normalizată	float $[0, 1]$	ora curentă împărțită la 24
17	relativeDistanceRank	float $[0, 1]$	proporția candidaților mai apropiați; 0 = cel mai apropiat
18	interestMatchScore	float $[0, 1]$	proporția categoriilor comune între utilizator și obiectiv
19	isPartOfRoute	binar	obiectivul face parte din ruta curentă
20	routeStopNormalized	float $[0, 1]$	poziția opririi în rută; 0 = prima oprire, 1 = ultima
21	routeLengthNormalized	float $[0, 1]$	lungimea rutei normalizată la maxim 5 opriri

4.4.2 Antrenarea locală

Procedura de antrenare locală este implementată conform protocolului clientului din FedAvg [13]. Dispozitivul antrenează modelul pe buffer local de interacțiuni, și trimite ulterior către server doar diferența ponderilor față de modelul global. O rundă de antrenare se declanșează automat după strângerea a 15 interacțiuni în buffer - poate fi inițiată de utilizator dacă au fost strânse cel puțin 8 interacțiuni cu obiectivele - și presupune rularea a 5 epoci complete peste întregul set local.

Optimizarea se realizează prin coborâre pe gradient stocastică (SGD, *Stochastic Gradient Descent*) cu rata de învățare $\eta = 0,01$, care este aplicată pentru fiecare eșantion, și pentru funcția de pierdere se folosește eroarea medie pătratică (MSE, *Mean Squared Error*), cu formula actualizărilor ilustrată în ecuația 1. Gradientul acestor actualizări este limitat la norma L2 unitară, iar dacă norma depășește 1, gradientul este scalat proporțional. Procesul este diferit de limitarea diferenței ponderilor aplicată ulterior în pasul de DP.

$$w_{t+1} = w_t - \eta \cdot \frac{g_t}{\max(1, \|g_t\|_2)} \quad (1)$$

unde w_t reprezintă ponderile la pasul t , $\eta = 0,01$ este rata de învățare, g_t este gradientul erorii medii pătratice calculat per eșantion, iar $\|g_t\|_2$ este norma sa L2.

4.4.3 Agregarea asincronă și reducerea în funcție de vechime

Serverul agrează imediat fiecare actualizare primită, fără a aștepta alți clienți, indiferent dacă este vorba de a aștepta toți utilizatorii sau doar de un grup mai mic, procesul de învățare de pe dispozitiv și apoi agregare reprezentând o rundă. Clientul trimite către server runda la care a descărcat modelul global (`client_round`), iar *staleness-ul* reprezintă câte runde au trecut de când clientul a primit modelul actualizat. Factorul de reducere [17], prezentat în ecuația 2, scalează numărul efectiv de eșantioane. Modelul global este ponderat cu de 4 ori mai multe eșantioane virtuale, astfel încât contribuția unui client să nu poată depăși singur 20% din agregare.

Funcția de reducere este ilustrată în figura 5, unde se observă că la *staleness*=0 clientul contribuie integral, la *staleness*=1 în proporție de 50%, iar la *staleness*=9 în proporție de doar 10%. Alegerea folosirii algoritmului se justifică prin următorul caz: dacă doi - sau mai mulți - clienți descarcă modelul global în același timp și unul termină antrenarea mai repede, serverul ajunge la runda următoare înainte ca al doilea să trimită actualizarea. Prin urmare, discountul penalizează proporțional contribuția celui rămas în urmă cu varianta modelului global.

$$\text{discount}(s) = \frac{1}{1+s}, \quad n_{\text{eff}} = \max(1, \lfloor n \cdot \text{discount}(s) \rfloor) \quad (2)$$

unde $s = \text{current_round} - \text{client_round}$ este vechimea actualizării, n este numărul de eşantioane locale ale clientului, iar n_{eff} este numărul efectiv folosit la agregare.

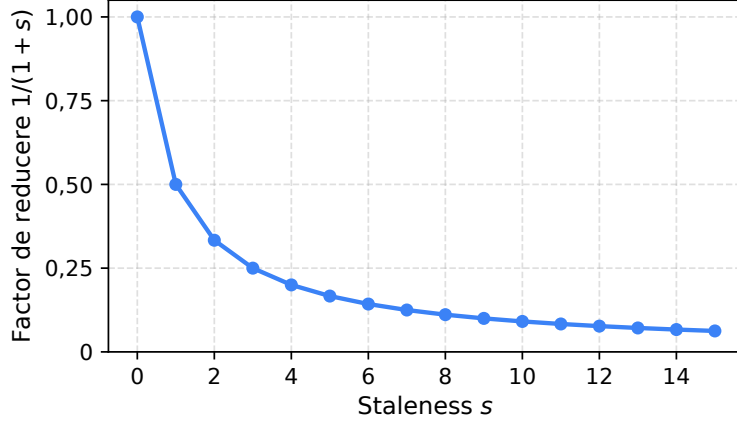


Figura 5: Factorul de reducere FedAsync în funcție de *staleness*.

4.4.4 Limitarea gradientilor

Înainte de agregare, serverul calculează variația ponderilor față de modelul global, denumită și delta. Dacă norma L2 a deltei depășește pragul $\tau = 1,0$, aceasta este rescalată proporțional conform ecuației 3 [14], după care actualizarea rescalată este trecută prin agregarea ponderată FedAvg. Pentru un client care antrenează în mod normal, delta are o normă mică și factorul de scalare este 1, deci, actualizarea trece nemodificată. Limitarea intervine doar în cazul în care delta este foarte mare față de ce ar produce o antrenare obișnuită.

Rolul principal al acestui tip de limitare este de a reduce influența actualizărilor extreme. Un client malițios care evaluează totul adversarial [12] produce o deltă cu normă mare, dar aceasta este redusă la τ înainte de agregare, limitând influența sa pentru fiecare coordonată de pondere, însă limitarea deltei este și o condiție necesară pentru garantarea DP din secțiunea următoare.

$$\tilde{\delta} = \delta \cdot \min\left(1, \frac{\tau}{\|\delta\|_2}\right) \quad (3)$$

unde $\delta = w_{\text{client}} - w_{\text{global}}$ este variația ponderilor față de modelul global, $\|\delta\|_2$ este norma sa L2, iar $\tau = 1$ este pragul de limitare.

4.4.5 Confidențialitate diferențială

Perturbarea prin zgomot nu se face central, zgomotul Gaussian fiind adăugat pe dispozitivul clientului, nu pe server, conform mecanismului NbAFL (*Noising before Model Aggregation Federated Learning*) [16]. Imediat după etapa de antrenare locală, fiecare coordonată a deltei este limitată la intervalul $[-\tau, \tau]$, și abia peste valorile rezultate se suprapune zgomotul. În felul acesta, la server ajung ponderile cu zgomotul deja aplicat, din care actualizarea reală nu mai poate fi reconstituită niciodată.

Limitarea făcută coordonată cu coordonată la acest pas nu trebuie confundată cu limitarea normei L2 discutată la secțiunea anterioară: aceea tratează delta ca pe un singur vector și are rolul de a apăra modelul de actualizările adversariale [12], pe când cea de față ține sensibilitatea mărginită, condiție care face ca zgomotul adăugat să aibă efect real asupra confidențialității [15]. Cât despre intensitatea lui, valoarea aleasă (0,1) reprezintă un echilibru asumat între confidențialitate și utilitatea modelului, pe care îl analizez în secțiunea de evaluare (Capitolul 6).

$$\tilde{w} = w_{\text{global}} + \text{clip}(\delta, \tau) + \mathcal{N}(0, (\sigma \cdot \tau)^2) \quad (4)$$

unde $\delta = w_{\text{trained}} - w_{\text{global}}$ este variația ponderilor, $\tau = 1$ este pragul de limitare, iar $\sigma = 0,1$ este multiplicatorul de zgomot.

4.4.6 Controlul concurenței

Citirea modelului global, agregarea și rescrierea lui nu alcătuiesc o operație atomică în mod implicit. Dacă doi clienți trimit actualizări în același timp, se poate ajunge ca ambii să citească modelul la aceeași versiune, să-și calculeze delta fiecare, independent, dar să-și suprascrie reciproc rezultatul agregării. Pentru a preveni acest caz, am implementat pentru server un *lock* Redis bazat pe SET NX PX: un token unic care este scris în Redis sub cheia `f1:agg_lock`, doar dacă aceasta nu există deja, ceea ce asigură că un singur client poate intra în etapa de agregare la un moment dat. Eliberarea *lock-ului* se face printr-un script Lua rulat atomic, care mai întâi verifică că token-ul aparține clientului curent pentru a evita ștergerea eronată a unui *lock* deținut de alt client.

Dacă *lock-ul* nu poate fi obținut, serverul mai încearcă de 9 ori, la intervale de câte 0,3 secunde, iar dacă tot nu reușește, întoarce o eroare. *Lock-ul* expiră de la sine după 5 secunde, în cazul în care serverul devine indisponibil în mijlocul agregării și *lock-ul* rămâne blocat. Am redat fluxul întreg în figura 6.

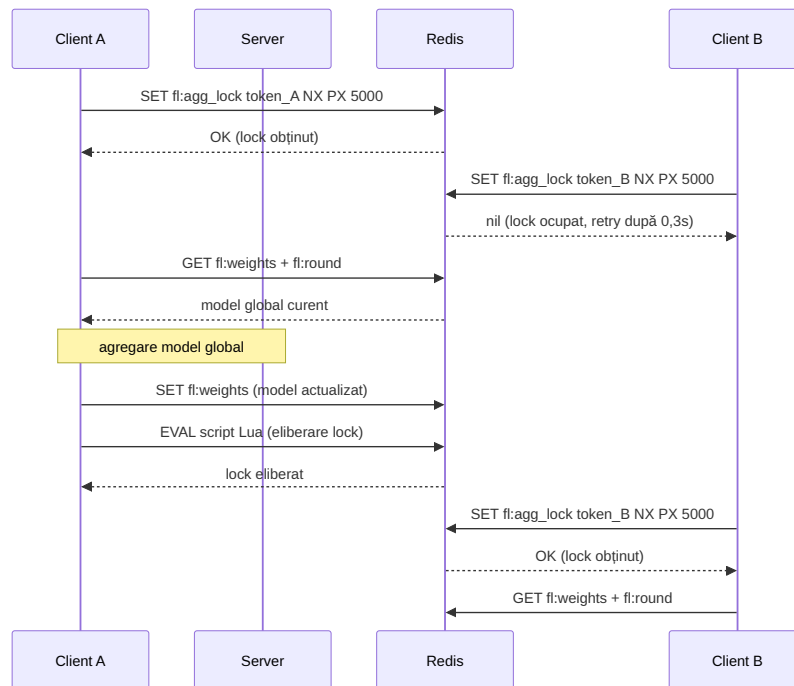


Figura 6: Controlul concurenței prin lock Redis.

4.5 Fluxul de generare a rutelor

Generarea unei rute cuprinde, mai întâi, locația curentă a utilizatorului și se ajunge un traseu ordonat de obiective din proximitate, alcătuit astfel încât să respecte preferințele legate de interese și un buget alocat de maxim 300 de minute. Calculul rulează pe server, în `route_service.py`, și este schițat în figura 7. În prima etapă se strâng toate obiectivele turistice aflate pe o rază de 10 km și le atribuie un scor inițial care combină scorul FL (în proporție de 80%) și proximitatea față de locația de start (în proporție de 20%). Dacă modelul global nu este disponibil în momentul respectiv, locul scorului FL este preluat de o fracție de potrivire a intereselor, fiind dedusă direct din categoriile obiectivului. Primele 10 locații - dacă există în apropiere - cu cel mai mare scor rămân drept candidați - suficiente pentru un traseu de cel mult 5 opriri.

Traseul ales să fie construit efectiv cu ajutorul unui algoritm de tip vecin cel mai apropiat: la fiecare pas, se alege obiectivul cel mai aproape de poziția curentă, însă cu o corecție care „scurtează” distanța față de candidații ale căror categorii se potrivesc intereselor utilizatorului - permițând unui obiectiv relevant, dar mai îndepărtat, să poată fi ales înaintea unuia neutru din imediata apropiere. Pentru fiecare oprire se estimează și o durată de vizitare. Aceasta pleacă de la un timp de bază specific tipului de obiectiv (de la 15 minute pentru o piață publică până la 70 de minute pentru un muzeu) și este scalată între 75% și 125% în funcție de scorul FL al opririi. Scorul FL se calculează cu vectorul de caracteristici menționat anterior, care include și poziția obiectivului în traseu. De altfel, același mecanism de scorare FL se află

și în spatele categoriei de potrivire afișate atunci când utilizatorul deschide un obiectiv de pe hartă.

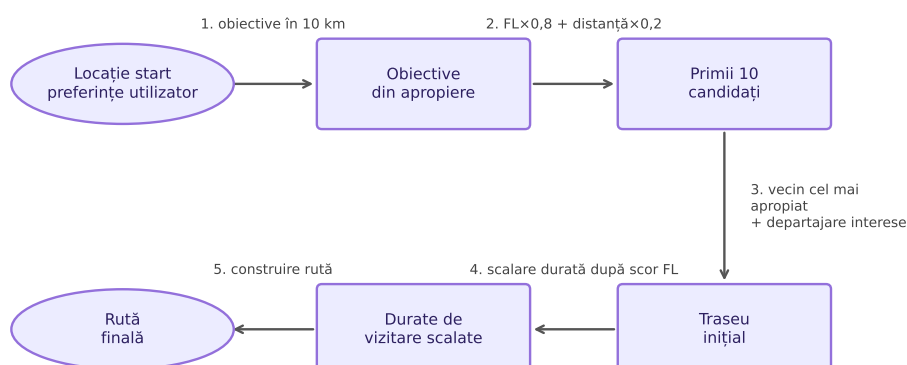


Figura 7: Fluxul de generare a rutelor.

4.6 Proiectarea elementelor de gamificare

Gamificarea se sprijină pe sistemul de *quest-uri* asociate obiectivelor turistice și culturale - sarcini scurte pe care utilizatorul le poate accepta și duce la capăt doar la fața locului, fiind răsplătit cu puncte la completare. Un *quest* poate fi adăugat atât de un administrator sau de o persoană care a creat obiectivul asociat pe hartă - și va fi disponibil imediat altor utilizatori, cât și de o persoană oarecare - în acest caz fiind nevoie ori de aprobarea unui administrator, ori de votul a cinci utilizatori fără rol de administrator. În acest fel, se încurajează contribuirea comunității în cadrul aplicației pentru a crește interactivitatea cu obiectivele, într-un meniu în care pot vedea toate *quest-urile* și poveștile trimise care au nevoie de aprobare. În plus, conform principiului confidențialității pe care se bazează CultureQuest, starea de finalizare a unui *quest* nu ajunge pe server, ci este gestionată exclusiv pe dispozitivul utilizatorului.

Fiecare interacțiune cu un obiectiv lasă în urmă un scor de *engagement* al turistului cu obiectivul, care reprezintă ținta de antrenament a modelului FL pentru acea interacțiune. Valoarea scorului crește odată cu „intensitatea interacțiunii”: deschiderea fișei unui obiectiv generează valoarea 0,3, adăugarea la rută a obiectivului sau un răspuns greșit la un *quest* aduce un scor de 0,5, iar un răspuns corect 0,9. Așadar, scorul maxim generat prin aceste interacțiuni va deveni scorul asociat obiectivului, și folosit la etapa de *backpropagation*. Evaluările cu stele sunt convertite după formula $n/5$, producând valori între 0,2 și 1 în funcție de numărul de stele lăsat de utilizator în cadrul recenziei, dar, în acest caz, valoarea rezultată suprascrie orice valoare produsă de interacțiunile anterioare, chiar dacă ar fi mai mare. Setul complet de etichete este redat în tabelul 7.

Tabelul 7: Etichete de *engagement*.

Eveniment	Etichetă
Fișa obiectivului deschisă	0,3
Obiectiv adăugat la rută	0,5
Quest: răspuns incorect	0,5
Navigare către obiectiv pornită	0,6
Quest propus de utilizator	0,80
Poveste trimisă	0,85
Quest: răspuns corect	0,9
Evaluare cu n stele	$n/5$

4.7 Harta și navigarea

Harta aplicației integrează tehnica de *geofencing*, implementată manual pe baza fluxului de poziții GPS furnizat de pachetul *geolocator*¹. De fiecare dată când poziția se actualizează, serviciul de proximitate recalculează distanțele până la obiectivele din jur și declanșează un eveniment de intrare sau ieșire în momentul în care utilizatorul traversează raza unui obiectiv. Lista obiectivelor monitorizate nu este însă reîncărcată la nesfârșit, ci abia după ce utilizatorul s-a deplasat cu mai mult de 200 de metri. Orientarea hărții este gestionată, la rândul ei, prin două mecanisme: în repaus, pachetul *flutter_compass*² rotește harta după busola magnetometrică și ignoră schimbările sub 2° pentru a nu prelua zgomotul senzorului; în modul de navigare activă, harta se rotește după direcția de deplasare raportată de GPS.

4.8 Arhitectura de confidențialitate

Arhitectura de confidențialitate a CultureQuest pornește de la ideea că datele trebuie separate explicit în funcție de locul de stocare. Tot ce ține de comportamentul concret al turistului - obiectivele vizitate, rating-urile date fiecărui obiectiv în parte, *quest-urile* finalizate și rutele salvate - rămân local, pe telefon, prin *SharedPreferences*, în directorul privat al aplicației, în așa fel încât alte aplicații să nu aibă acces la date, fiind o responsabilitate a *sandbox-ului* Android/iOS, iar spre server vin doar informații agregate sau anonimizate. Concret, se incrementează un contor global de vizite pentru fiecare obiectiv în locul primirii identității vizitatorului, respectiv se modifică suma și numărul de rating-uri în locul evaluării de către vizitator. Pe server, totul se rezumă la a se păstra doar datele necesare funcționării aplicației: numele complet, punctajul total, numărul de *quest-uri* completate și interesele selectate. Aceeași logică se aplică și componentei FL: serverul primește doar delta-ul ponderilor obținut

¹Pachet Flutter pentru accesul la localizare GPS. <https://pub.dev/packages/geolocator> (accesat 19.06.2026)

²Pachet Flutter pentru citirea busolei magnetometrice. https://pub.dev/packages/flutter_compass (accesat 19.06.2026)

după antrenamentul local, peste care s-a adăugat deja zgomot Gaussian. Distribuția completă este redată în tabelul 8, iar modul în care sunt implementate mecanismele de confidențialitate este detaliat în Capitolul 5.

Arhitectura actuală are însă și o limitare: serverul primește delta-ul fiecărui client în mod individual, așa că se pot distinge, în principiu, contribuțiile individuale. Protocolul de agregare securizată [5] ar elimina acest risc, făcând ca serverul să vadă doar suma actualizărilor, nu și delta-urile individuale. Numai că SecAgg este incompatibil cu configurația asincronă pe care se bazează platforma. Protocolul cere ca un set fix de clienți să fie activi în același timp pentru schimbul de chei criptografice, ceea ce contravine modelului FedAsync în care clienții se conectează independent, fiecare la momentul lui. Mai mult, discountul de *staleness* aplicat pentru fiecare client la agregare are nevoie de acces la delta-urile individuale, nu doar la suma lor.

Tabelul 8: Distribuția datelor în funcție de locul de stocare.

Date locale (SharedPreferences)	Date pe server	Ce nu se stochează pe server
Lista obiectivelor vizitate	Numele utilizatorului	Istoricul locației
Rating-uri individuale pentru fiecare obiectiv	Puncte totale acumulate	Care obiective au fost vizitate și când
Quest-uri completate (care și cum)	Număr total de quest-uri completate	Rating-urile individuale pentru fiecare obiectiv
Rute salvate	Interese selectate	Care quest-uri au fost completate
Identitatea autorului recenziei	-	Autorul fiecărei recenzii sau povești

5 DETALII DE IMPLEMENTARE

5.1 Implementarea aplicației mobile

5.1.1 Serviciul client de învățare federată

5.1.2 Adăugarea de zgomot pentru asigurarea confidențialității diferențiale

5.1.3 Bufferul local de interacțiuni și persistența datelor

5.1.4 Stocarea temporară a obiectivelor pentru funcționarea offline

5.1.5 Algoritmul de generare a rutelor

5.1.6 Sistemul de misiuni și tehnologia de *geofencing*

5.2 Implementarea serviciilor *backend*

5.2.1 Serviciul de agregare

5.2.2 Punctele de acces API

5.2.3 Sistemul de recenzii și moderarea conținutului

5.3 Fluxul etapei de pre-antrenare

5.4 *Framework-ul* pentru simularea procesului de învățare federată

5.5 Dificultăți întâmpinate în procesul de implementare

6 EVALUARE

6.1 Corectitudinea funcțională

6.2 Performanța modelului de învățare federată

6.2.1 Rezultatele etapei de pre-antrenare

6.2.2 Rezultatele simulării procesului de învățare federată

6.3 Evaluarea confidențialității și robusteții

6.4 Analiză comparativă și discuții

7 CONCLUZII

7.1 Concluzii generale

7.2 Direcții de cercetare și lucrări viitoare

BIBLIOGRAFIE

- [1] D. A. E. Acar, Y. Zhao, R. M. Navarro, M. Mattina, P. N. Whatmough, and V. Saligrama. Federated learning based on dynamic regularization. In *International Conference on Learning Representations (ICLR)*, 2021. doi:10.48550/arXiv.2111.04263.
- [2] G. Adomavicius and A. Tuzhilin. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6):734–749, 2005. doi:10.1109/TKDE.2005.99.
- [3] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary. Federated learning with personalization layers. *arXiv preprint arXiv:1912.00818*, 2019. doi:10.48550/arXiv.1912.00818.
- [4] K. Bonawitz, H. Eichner, N. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konecny, S. Mazzocchi, H. B. McMahan, T. Van Overveldt, D. Petrou, D. Ramage, and J. Roselander. Towards federated learning at scale: System design. In *Proceedings of the 2nd SysML Conference*, 2019. doi:10.48550/arXiv.1902.01046.
- [5] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017. doi:10.1145/3133956.3133982.
- [6] D. Gavalas, C. Konstantopoulos, K. Mastakas, and G. Pantziou. Mobile recommender systems in tourism. *Journal of Network and Computer Applications*, 39:319–333, 2014. doi:10.1016/j.jnca.2013.04.006.
- [7] P. Kairouz, H. B. McMahan, et al. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14(1–2), 2021. doi:10.1561/22000000083.
- [8] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh. Scaffold: Stochastic controlled averaging for federated learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. doi:10.48550/arXiv.1910.06378.
- [9] L. Li, Y. Fan, M. Tse, and K.-Y. Lin. A review of applications in federated learning. *Computers & Industrial Engineering*, 149:106854, 2020. doi:10.1016/j.cie.2020.106854.
- [10] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine*, 37(3), 2020. doi:10.1109/MSP.2020.2975749.

- [11] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2020. doi:10.48550/arXiv.1812.06127.
- [12] L. Lyu, H. Yu, and Q. Yang. Threats to federated learning: A survey. *arXiv preprint arXiv:2003.02133*, 2020. doi:10.48550/arXiv.2003.02133.
- [13] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017. doi:10.48550/arXiv.1602.05629.
- [14] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang. Learning differentially private recurrent language models. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.06963. doi:10.48550/arXiv.1710.06963.
- [15] R. Shokri, M. Stronati, C. Song, and V. Shmatikov. Membership inference attacks against machine learning models. In *IEEE Symposium on Security and Privacy (S&P)*, 2017. doi:10.1109/SP.2017.41.
- [16] K. Wei, J. Li, M. Ding, C. Ma, H. H. Yang, F. Farokhi, S. Jin, T. Q. S. Quek, and H. V. Poor. Federated learning with differential privacy: Algorithms and performance analysis. *IEEE Transactions on Information Forensics and Security*, 15, 2020. doi:10.1109/TIFS.2020.2988575.
- [17] C. Xie, S. Koyejo, and I. Gupta. Asynchronous federated optimization. *arXiv preprint arXiv:1903.03934*, 2019. doi:10.48550/arXiv.1903.03934.
- [18] Q. Yang, Y. Liu, T. Chen, and Y. Tong. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology*, 10(2):12, 2019. doi:10.1145/3298981.
- [19] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao. A survey on federated learning. *Knowledge-Based Systems*, 216:106775, 2021. doi:10.1016/j.knosys.2021.106775.

ANEXE

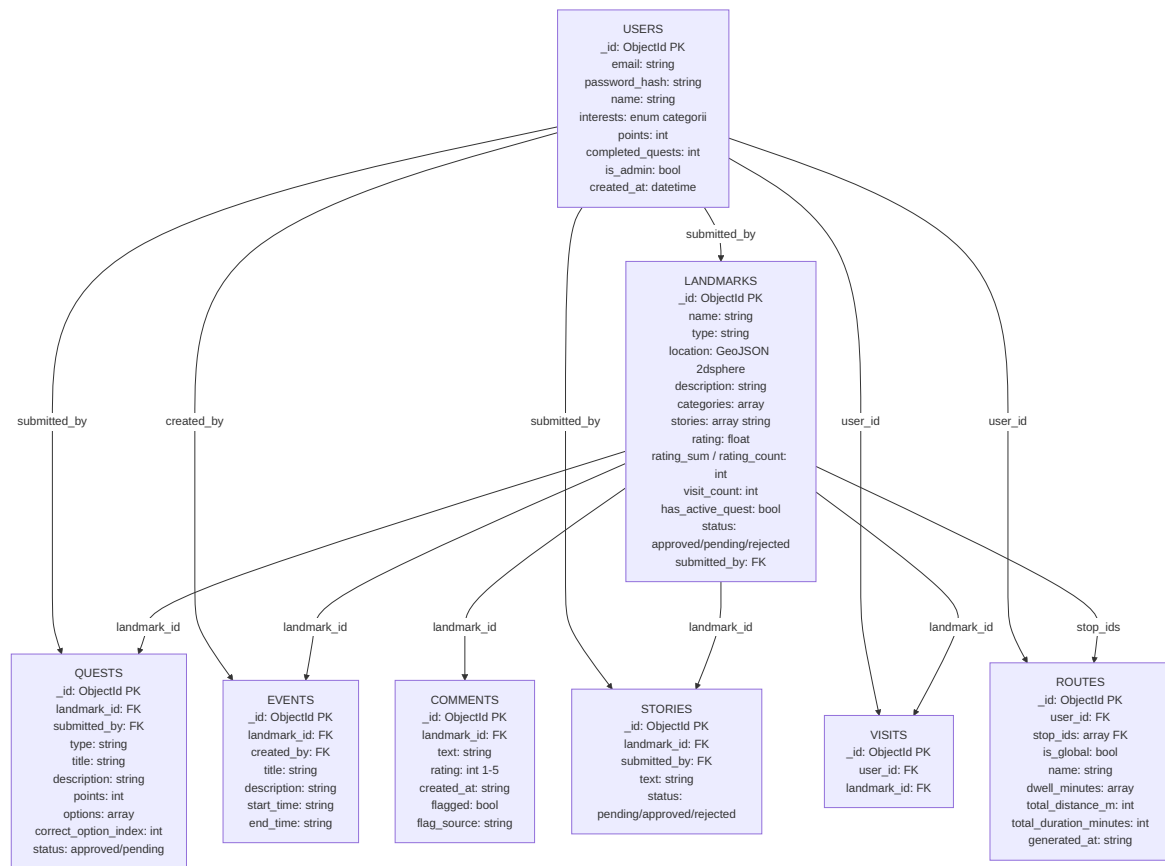


Figura 8: Modelul complet de date al aplicației CultureQuest: colecțiile MongoDB, câmpurile principale și relațiile dintre ele.

A EXTRASE DE COD